

高性能探索：从 Triton 算子到高性能通用算子 FlagGems



2025年08月17日

目录

01 FlagGems简介

02 Triton 算子开发示例

03 FlagGems特性

04 FlagGems contribute



<https://github.com/FlagOpen/FlagGems>



一个使用**OpenAI Triton**语言实现的高性能算子库

广覆盖

V3.0中实现186个算子，基本覆盖大模型需求
大部分是aten算子，包括一些重点和融合算子

高性能

60%+算子性能持平或由于原生实现

轻量级

多款硬件支持，涵盖NVIDIA与国产芯片
安装使用简便，无缝嵌入PyTorch

FlagGems算子库

北京智源人工智能研究院
BEIJING ACADEMY OF ARTIFICIAL INTELLIGENCE



联合开发

BAI
智源研究院

 **天数智芯**
Iluvatar CoreX

Cambricon
寒 武 纪

 **中科加禾**
— CORE SIGMA —

 **摩尔线程**
MOORE THREADS

META  **沐曦**

 **SILICONFLOW**

 **昆仑芯**
KUNLUNXIN

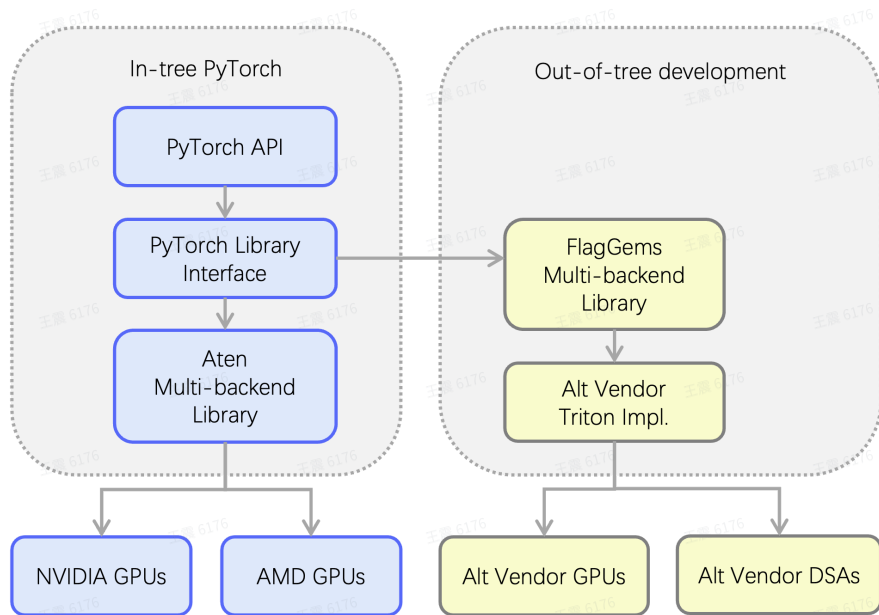
HYGON

- 即插即用

- 注册到PyTorch的分发机制
- 拦截对ATen算子的调用
- 无障碍替换CUDA算子实现

- 一次编写、随处编译

- 统一的算子库代码
- 具备Triton的后端均可编译
- 支持GPU与DSA等异构设备



- 在进程中永久启用

```
import torch
import flag_gems
flag_gems.enable()

M, N, K = 1024, 1024, 1024
A = torch.randn((M, K), dtype=torch.float16, device="cuda")
B = torch.randn((K, N), dtype=torch.float16, device="cuda")
C = torch.mm(A, B)
```

- 在进程局部临时启用

```
import torch
import flag_gems

M, N, K = 1024, 1024, 1024
A = torch.randn((M, K), dtype=torch.float16, device="cuda")
B = torch.randn((K, N), dtype=torch.float16, device="cuda")
with flag_gems.use_gems():
    C = torch.mm(A, B)
```

- 直接调用

```
import torch
import flag_gems

M, N, K = 1024, 1024, 1024
A = torch.randn((M, K), dtype=torch.float16, device="cuda")
B = torch.randn((K, N), dtype=torch.float16, device="cuda")
C = flag_gems.mm(A, B)
```

FlagGems部分算子清单

LinearAlgebra

BLAS	Reduction	
mm	cumsum	max
bmm	mean	argmax
linear*	sum	amax
matmul**	prod	min
addmm	var_mean	
mv		
outer		

NeuralNetwork

BasicMath

Fusion

Backward

* linear底层调用addmm实现

** matmul底层调用bmm, mm, mv实现

FlagGems部分算子清单

LinearAlgebra

NeuralNetwork

Activation	Normalization	Other
gelu	layer_norm*	dropout***
relu	group_norm**	triu
silu	vector_norm	cross_entropy_loss
softmax	rms_norm	native_dropout
log_softmax	native_layer_norm	
sigmoid	native_group_norm	

BasicMath

Fusion

Backward

* layer_norm底层调用native_layer_norm实现
** group_norm底层调用native_group_norm实现
*** dropout底层调用native_dropout实现

FlagGems部分算子清单

LinearAlgebra

NeuralNetwork

BasicMath

Mathmatical		Logical		Reduction
add	abs	bitwise_and	eq	all
div	exp	bitwise_not	ge	any
sub	reciprocal	bitwise_or	gt	
mul	rsqrt	__or__	le	
pow	neg	isinf	lt	
rsub	cos	isnan	ne	
clamp	sin			
	tanh			

Fusion

Backward

FlagGems部分算子清单

LinearAlgebra

NeuralNetwork

BasicMath

Fusion

Backward

Mathematical	Pointwise	Attention
skip_layernorm	silu_and_mul	rotary_position_embedding
skip_rmsnorm	gelu_and_mul	flash_attention
		flash_mla

FlagGems部分算子清单

LinearAlgebra

NeuralNetwork

BasicMath

Fusion

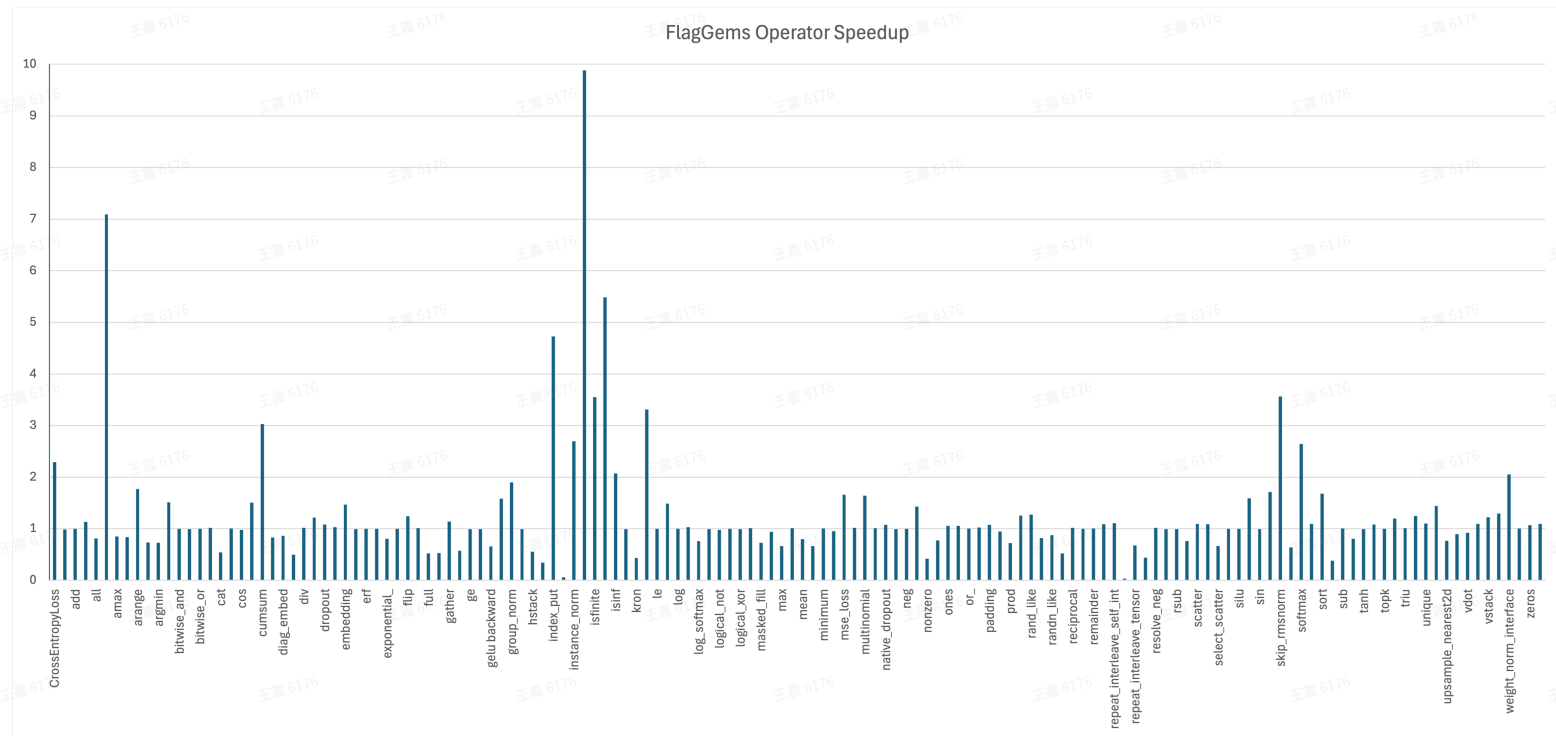
Backward*

Pointwise	Reduction	Other
native_dropout_backward	native_layer_norm_backward	outer_backward
threshold_backward	_softmax_backward_data	cross_entropy_loss_backward
silu_backward	_log_softmax_backward_data	
tanh_backward	native_group_norm_backward	
sigmoid_backward		

* Backward算子未计入总数

FlagGems与Aten Kernel性能对比

- FlagGems支持186个算子
- 63%的算子性能优于原生的CUDA算子



目录

01 FlagGems简介

02 Triton 算子开发示例

03 FlagGems特性

04 FlagGems contribute

Vector Add(1 tile/CTA)

1 tile/CTA

CTA 0	CTA 1	CTA 2	CTA 3	CTA 4
-------	-------	-------	-------	-------

TILE_SIZE

@triton.jit

```
def add_kernel(a_ptr, b_ptr, o_ptr, N, TILE_SIZE: tl.constexpr):
```

```
    pid = tl.program_id(0)
```

```
    offsets = pid * TILE_SIZE + tl.arange(0, TILE_SIZE)
```

```
    mask = offsets < N
```

```
    a = tl.load(a_ptr + offsets, mask=mask)
```

```
    b = tl.load(b_ptr + offsets, mask=mask)
```

```
    o = a + b
```

```
    tl.store(o_ptr + offsets, o, mask=mask)
```

```
def add(a, b):
```

```
    o = torch.empty_like(a)
```

```
    N = a.numel()
```

```
    TILE_SIZE = 128
```

```
    grid = (triton.cdiv(N, TILE_SIZE), 1, 1)
```

```
    add_kernel[grid](a, b, o, N, TILE_SIZE, num_warps=4)
```

```
    return o
```

kernel: 描述一个 **CTA** 的运算。
各个 CTA 之间的差别在于 program_id。

没有细化到每个 thread 做什么

wrapper: 指定 grid, 启动 kernel.

Vector Add(ntiles/CTA)

2 tiles/CTA

CTA 0	CTA 0	CTA 1	CTA 1	CTA 2	CTA 2
TILE_SIZE					

@triton.jit

```
def add_kernel(a_ptr, b_ptr, o_ptr, N, num_tiles_per_CTA, TILE_SIZE: tl.constexpr):
```

```
    pid = tl.program_id(0)
```

```
    program_offset = pid * num_tiles_per_CTA * TILE_SIZE
```

```
    * offsets = program_offset + tl.arange(0, TILE_SIZE)
```

```
    for i in range(num_tiles_per_CTA):
```

```
        mask = offsets < N
```

```
        a = tl.load(a_ptr + offsets, mask=mask)
```

```
        b = tl.load(b_ptr + offsets, mask=mask)
```

```
        o = a + b
```

```
        tl.store(o_ptr + offsets, o, mask=mask)
```

```
        offsets += TILE_SIZE
```

```
def add(a, b):
```

```
    o = torch.empty_like(a)
```

```
    N = a.numel()
```

```
    num_tiles_per_CTA = 2
```

```
    TILE_SIZE = 128
```

```
    grid = (triton.cdiv(N, num_tiles_per_CTA * TILE_SIZE), 1, 1)
```

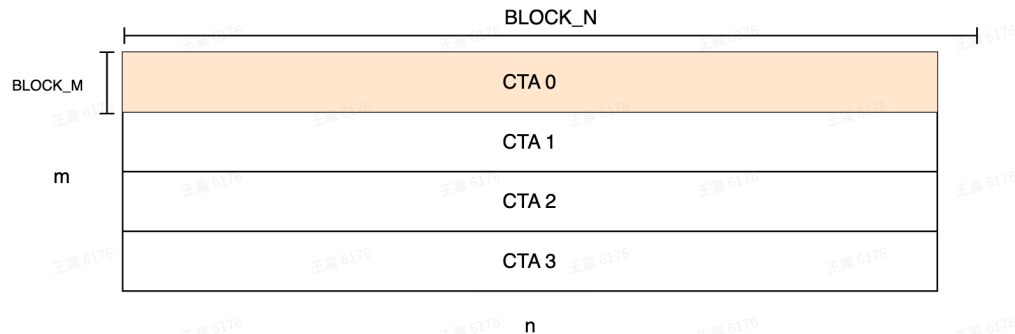
```
    add_kernel[grid](a, b, o, N, num_tiles_per_CTA, TILE_SIZE)
```

```
    return o
```

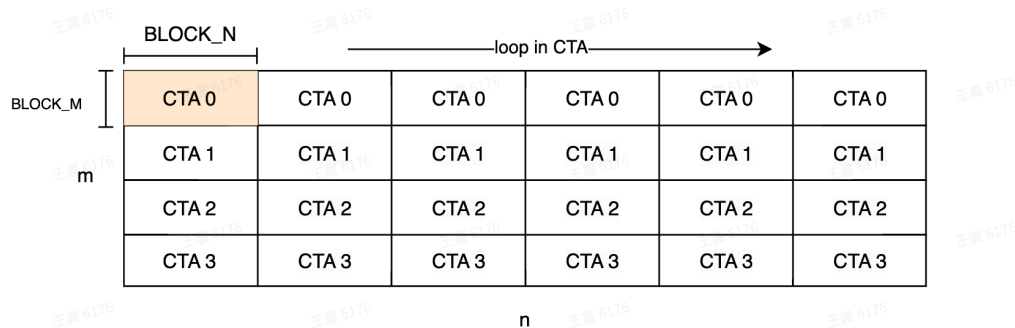
Reduce Sum

形状 (m, n) 的矩阵，沿着最后一个维度求和。

任务划分方案 1:
一个 CTA 处理若干行，作为一个 TILE 处理完，不使用循环



任务划分方案 2:
一个 CTA 处理若干行，每次处理 $(BLOCK_M, BLOCK_N)$ 的数据块 CTA 内包含循环

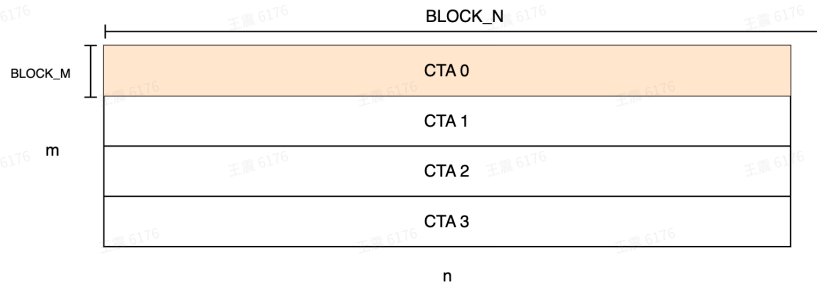


Reduce Sum(TILE 覆盖整行)

@triton.jit

```
def sum_kernel(output_ptr, input_ptr, M, N,  
               BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr,  
               ):  
    pid = tl.program_id(0)  
    *  
    m_offset = pid * BLOCK_M + tl.arange(0, BLOCK_M)  
    n_offset = tl.arange(0, BLOCK_N)  
    offset = m_offset[:, None] * N + n_offset[None, :]  
  
    m_mask = m_offset < M  
    n_mask = n_offset < N  
    mask = m_mask[:, None] & n_mask[None, :]  
  
    inp = tl.load(input_ptr + offset, mask=mask, other=0)  
    out = tl.sum(inp, axis=1)
```

```
tl.store(output_ptr + m_offset, out, mask=m_mask)
```



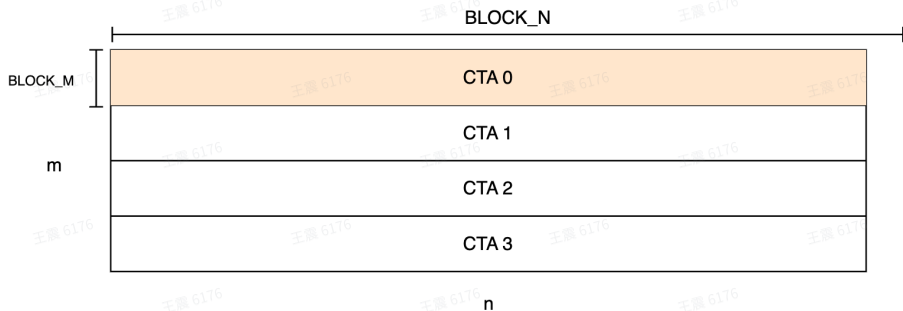
一个 CTA 计算 `BLOCK_M` 行

使用 2D tensor: 注意 broadcast 语法
，和 numpy/torch 是类似的。

tl 有 cooperative primitive

Reduce Sum (TILE 覆盖整行)

一个 CTA 计算 BLOCK_M 行



```
def sum(x):  
    m, n = x.shape  
    out = torch.empty(x.shape[:-1], dtype=x.dtype, device=x.device)
```

```
    BLOCK_M = 4  
    BLOCK_N = triton.next_power_of_2(n)  
    grid = triton.cdiv(m, BLOCK_M), 1, 1
```

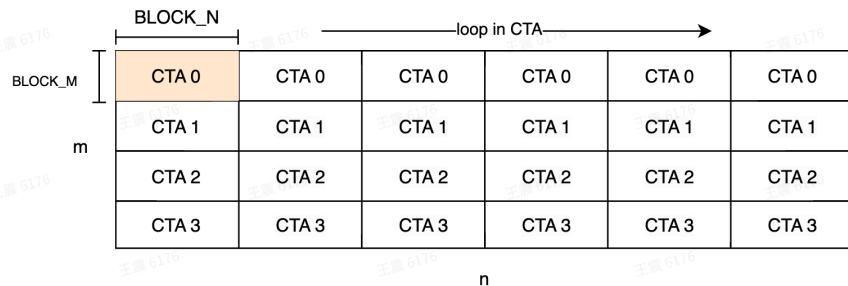
```
    sum_kernel[grid](out, x, m, n, BLOCK_M, BLOCK_N)  
    return out
```

Q: 怎么保证一个数据块能加载完一整行?

tl.arange 要求 size 需要是 2 的幂。

Reduce Sum (CTA 内循环)

```
@triton.jit
def sum_kernel(output_ptr, input_ptr,
              M, N,
              BLOCK_M: tl.constexpr,
              BLOCK_N: tl.constexpr,
              ):
    pid_m = tl.program_id(0)
    m_offset = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    m_mask = m_offset < M
    *
    acc = tl.zeros([BLOCK_M, BLOCK_N], dtype=tl.float32)
    for start_n in range(0, N, BLOCK_N):
        n_offset = start_n + tl.arange(0, BLOCK_N)
        n_mask = n_offset < N
        *
        offset = m_offset[:, None] * N + n_offset[None, :]
        mask = m_mask[:, None] & n_mask[None, :]
        inp = tl.load(input_ptr + offset, mask=mask, other=0)
        acc += inp
    *
    out = tl.sum(acc, axis=1)
    tl.store(output_ptr + m_offset, out, mask=m_mask)
```



数据块大小为 (BLOCK_M, BLOCK_N)，一个 CTA 处理若干块，直到一整行完结。

块之间 add, 最后再 sum

kernel launch 任务划分
grid = triton.cdiv(m, BLOCK_M), 1, 1

Reduce Sum: two pass reduction

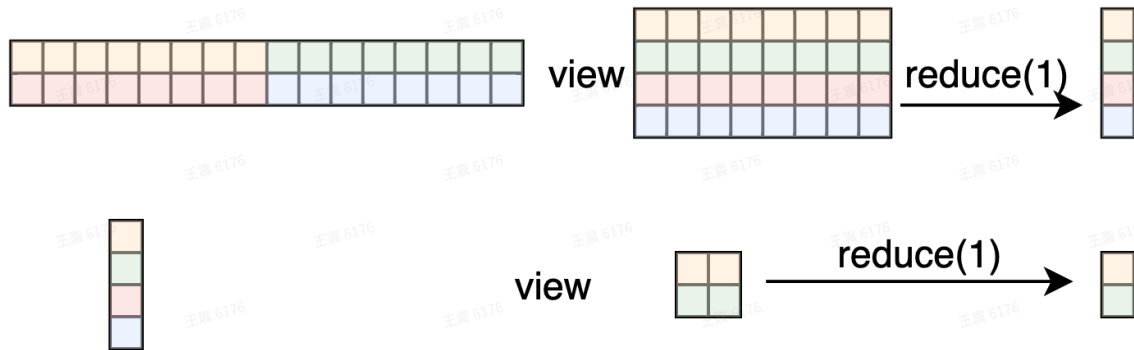
(m, n) 沿 axis 1 求和, n 较大, m 相对较小 (仅利用 m 维度上的并行性无法充分利用 GPU 的 SM)。使用两次 reduce, 调用两个 Kernel.

step1:

输入视为 $(m * n / K, K)$,
沿 axis 1 求和 $\rightarrow (m * n / K,)$

step2:

结果视为 $(m, n / K)$,
沿 axis 1 求和, $\rightarrow (m,)$



(M, K) 矩阵A乘行主序的 (K, N) 矩阵B，输出到 (M, N) 矩阵C。

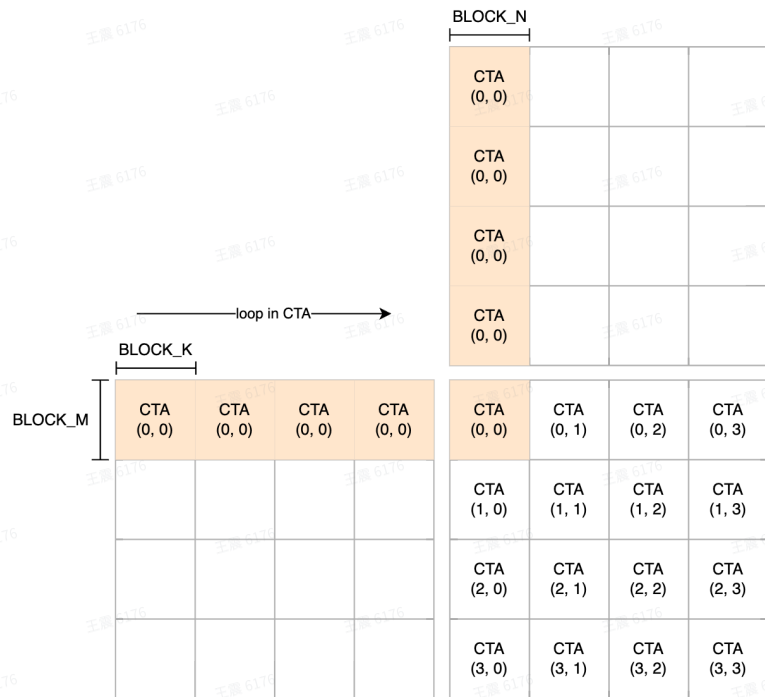
```
@triton.jit
def mm_kernel(
    A, B, C,
    M, N, K,
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
):
    # 2d -grid
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)

    *
    offsets_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    mask_m = offsets_m < M
    offsets_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    mask_n = offsets_n < N

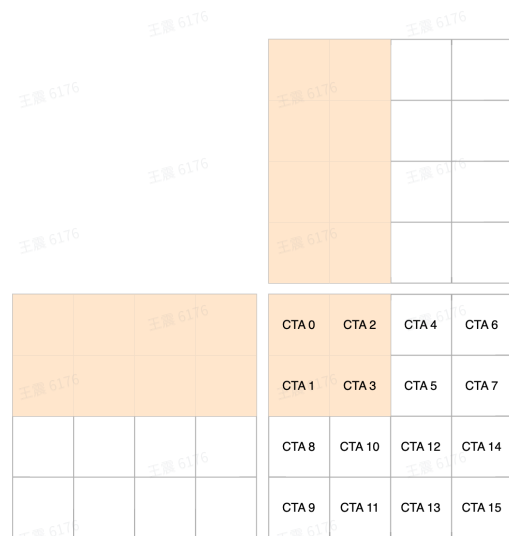
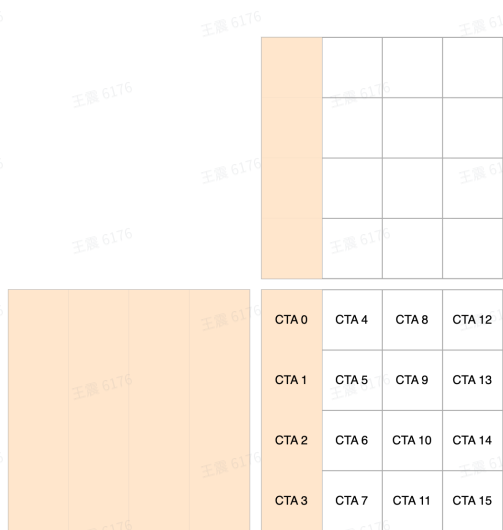
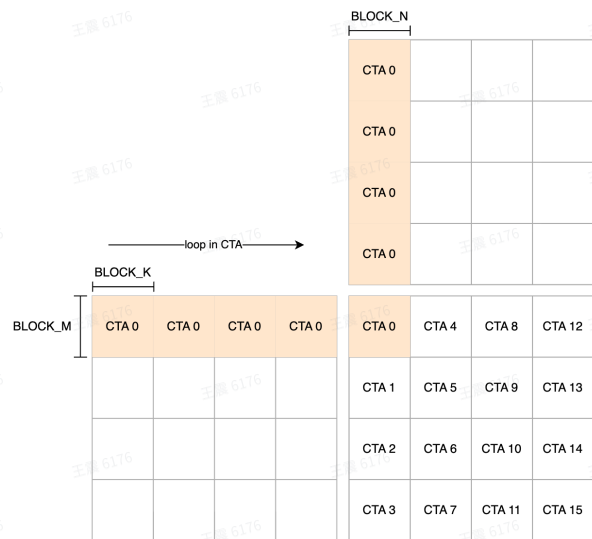
    *
    # 2d tile
    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
    for start_k in range(0, K, BLOCK_K):
        offsets_k = start_k + tl.arange(0, BLOCK_K)
        mask_k = offsets_k < K
        a_ptrs = A + offsets_m[:, None] * K + offsets_k[None, :]
        mask_a = mask_m[:, None] & mask_k[None, :]
        b_ptrs = B + offsets_k[:, None] * N + offsets_n[None, :]
        mask_b = mask_k[:, None] & mask_n[None, :]

        a = tl.load(a_ptrs, mask=mask_a, other=0)
        b = tl.load(b_ptrs, mask=mask_b, other=0)
        acc += tl.dot(a, b)

    *
    c_ptrs = C + offsets_m[:, None] * N + offsets_n[None, :]
    mask_c = mask_m[:, None] & mask_n[None, :]
    tl.store(c_ptrs, acc, mask=mask_c)
```



grid = triton.cdiv(M, BLOCK_M), triton.cdiv(N, BLOCK_N), 1



CTA 的编号和启动顺序是有关的。
xyz, x 变动最快。所以前面的写法实际相当于列主序的方式排列 CTA.

读 16 + 4 块, 写 4 块

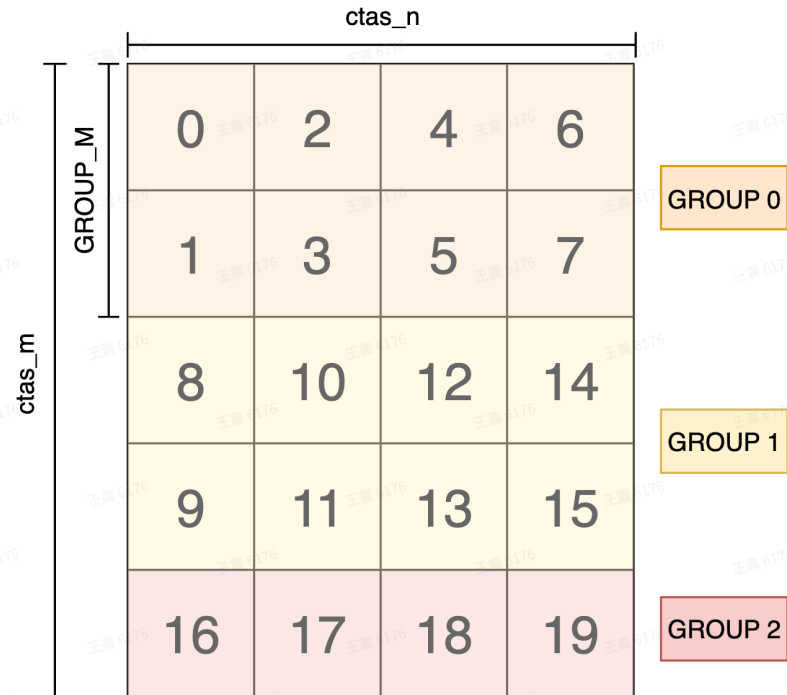
读 8 + 8 块, 写 4 块
提升 cache 命中率

重排 CTA

```
ctas_m, ctas_n = tl.cdiv(M, BLOCK_M), tl.cdiv(N, BLOCK_N)
ctas_per_group = GROUP_M * ctas_n
*pid = tl.program_id(0)
group_id = pid // ctas_per_group
inner_group_id = pid % ctas_per_group
*start_pid_m = group_id * GROUP_M
group_m_size = min(ctas_m - start_pid_m, GROUP_M)
pid_m = start_pid_m + inner_group_id % group_m_size
pid_n = inner_group_id // group_m_size
```

其他 CTA 层任务划分的优化手段:

- 调节 BLOCK_M, BLOCK_N, BLOCK_K 的大小
- split-k, K 维度划分, 用不同的 CTA 来计算, 当 M, N 小而 K 很大的时候适用。用 atomic 的方式累加结果。
- 用类似 two-pass reduction 的方式启动多个 Kernel 分阶段求和。



triton JIT Function 的参数中有一部分是编译器常量参数，用 **tl.constexpr** 标记，这部分参数类似模板参数，直接编译进 kernel 里，不再出现在 kernel 参数列表。

- 比如 mm_kernel 里的 BLOCK_M, BLOCK_N, BLOCK_K, GROUP_M.

还有部分不出现在 JIT Function 定义中，比如

- num_warps: 决定 CTA 中的 warp 数；
- num_stages: 决定 for 循环软流水 stage 数，影响共享内存的使用量。

但可以在调用 jit function 的时候传。

这些组成了一组配置 triton.Config. 用于 Tuning.

Matmul: Tuning

```
@triton.autotune(  
    configs=[  
        triton.Config({"BLOCK_M": 128, "BLOCK_N": 128, "BLOCK_K": 64, "GROUP_M": 16}, num_stages=3, num_warps=8),  
        triton.Config({"BLOCK_M": 128, "BLOCK_N": 64, "BLOCK_K": 32, "GROUP_M": 8}, num_stages=4, num_warps=4),  
        triton.Config({"BLOCK_M": 128, "BLOCK_N": 128, "BLOCK_K": 32, "GROUP_M": 4}, num_stages=4, num_warps=4),  
        triton.Config({"BLOCK_M": 128, "BLOCK_N": 64, "BLOCK_K": 32, "GROUP_M": 2}, num_stages=4, num_warps=4)  
    ],  
    key=["M", "N", "K"]  
)
```

cofigs: 备选配置

key: 指明 jit function 的哪些运行时参数影响配置选取

```
@triton.jit  
def mm_kernel(  
    A, B, C,  
    M, N, K,  
    BLOCK_M: tl.constexpr,  
    BLOCK_N: tl.constexpr,  
    BLOCK_K: tl.constexpr,  
    GROUP_M: tl.constexpr,  
):  
  
def mm(a, b):  
    ....  
    def grid(args):  
        return triton.cdiv(M, args["BLOCK_M"]) * triton.cdiv(N, args["BLOCK_N"]), 1, 1  
    mm_kernel[grid](  
        a, b, c,  
        M, N, K,  
    )  
    return c
```

目录

01 FlagGems简介

02 Triton 算子开发示例

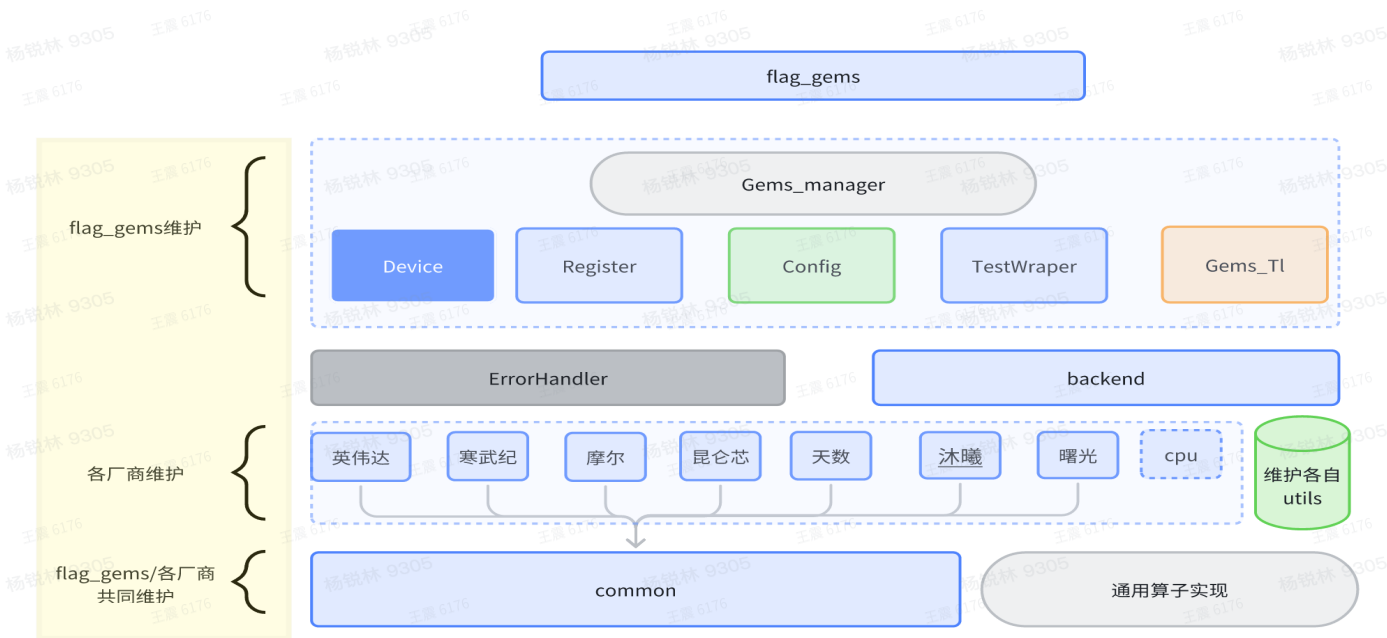
03 FlagGems特性 ---- 多后端

04 FlagGems contribute

多后端支持(Multi-Backend) - 跨厂商协同发展，加快形成统一开放的开源生态

- 跨厂商协同布局, 兼容不同厂商Triton版本, 异构芯片部署敏捷化、便捷化
- 多硬件一体衔接, 统筹敏捷开发与高效维护
 - PyTorch功能函数接口、TritonLanguage原语接口等标准化
 - 运行设备保护、张量设备指定
 - 错误信息管理、异常处理
 - 导入模块时自动进行后端检测
- 定制调参空间 & 定制启发式算法 & 定制算子实现:
 - 允许后端根据硬件特性调整配置参数与算子代码
 - 允许后端禁用性能低洼算子、不予注册
 - 允许用户手动选择禁用算子

整体上，在考虑可维护性、可扩展性、易用性等的前提下，核心思想是“一次注册，多处使用”构建一个通用的后端设备接口层，使得不同的硬件后端可以通过统一的接口与flag_gems算子库进行交互，在具体接口内部进行不同厂商的实现分发，这样，用户只需一次注册所需的算子，就可以在不同的硬件平台上无缝切换和使用



vendor	state	float16	float32	bfloat16
aipu	✓ (Partial support)	✓	✓	✓
ascend	✓ (Partial support)	✓	✓	✓
cambricon	✓	✓	✓	✓
hygon	✓	✓	✓	✓
iluvatar	✓	✓	✓	✓
kunlunxin	✓	✓	✓	✓
metax	✓	✓	✓	✓
mthreads	✓	✓	✓	✓
nvidia	✓	✓	✓	✓
arm(cpu)	⚠			
tsingmicro	⚠			

目前已经支持9种后端，2种后端正在支持中

目录

01 FlagGems简介

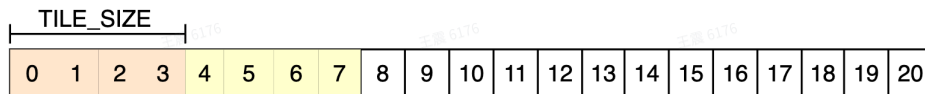
02 Triton 算子开发示例

03 FlagGems特性 ---- 代码生成

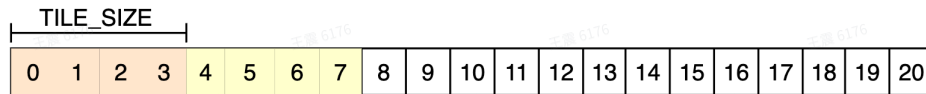
04 FlagGems contribute

如何用 triton 写一个功能较全的 pointwise 算子。从最简单的 vector add 开始。

```
@triton.jit
def add_kernel(a_ptr, b_ptr, o_ptr, N, TILE_SIZE: tl.constexpr):
    pid = tl.program_id(0)
    offsets = pid * TILE_SIZE + tl.arange(0, TILE_SIZE)
    mask = offsets < N
    *
    a = tl.load(a_ptr + offsets, mask=mask)
    b = tl.load(b_ptr + offsets, mask=mask)
    o = a + b
    *
    tl.store(o_ptr + offsets, o, mask=mask)
    *
def add(a, b):
    o = torch.empty_like(a)
    N = a.numel()
    TILE_SIZE = 512
    grid = triton.cdiv(N, TILE_SIZE)
    add_kernel[grid](a, b, o, N, TILE_SIZE)
    return o
```



```
@triton.jit
def add_kernel(a_ptr, b_ptr, o_ptr, N, TILE_SIZE: tl.constexpr):
    pid = tl.program_id(0)
    offsets = pid * TILE_SIZE + tl.arange(0, TILE_SIZE)
    mask = offsets < N
    * a = tl.load(a_ptr + offsets, mask=mask)
    * b = tl.load(b_ptr + offsets, mask=mask)
    * o = a + b
    * tl.store(o_ptr + offsets, o, mask=mask)
def add(a, b):
    o = torch.empty_like(a)
    N = a.numel()
    TILE_SIZE = 512
    grid = triton.cdiv(N, TILE_SIZE)
    add_kernel[grid](a, b, o, N, TILE_SIZE)
    return o
```



上述代码的问题：

- 当 tensor 形状不同，需要 broadcast 是无法处理；
- tensor 数据不连续，比如 permute 过，或者数据是分散的情况不能处理。

原因来自其加载和保存部分的地址偏移量的计算，就是预设计算首地址后连续的一片内存区域。

解法一: 使用 wrapper 处理 non-contiguous

```
def add_func_naive(x, y):  
    # supports broadcasting now  
    x, y = torch.broadcast_tensors(x, y)  
  
    # support non-contiguous now  
    x = x.contiguous()  
    y = y.contiguous()  
    o = torch.empty(x.shape, dtype=x.dtype, device=x.device)  
    ** TILE_SIZE = 512  
    num_warps = 4  
    N = x.numel()  
    grid = triton.cdiv(N, TILE_SIZE), 1, 1  
    add_kernel[grid](x, y, o, N, TILE_SIZE, num_warps=num_warps)  
    return o
```

使用 wrapper 调用 `torch.broadcast_tensor` 来处理 broadcasting, 使用 `contiguous` 来处理数据不连续的问题。

使用 wrapper 处理 broadcast 和非连续的输入 tensor, 性能不一定好, 这个过程本身就可能有一次数据 copy.

解法二：支持 stride

假设 a, b 来自两个大的 tensor 上的不连续切片。

```
a = torch.randn(200)[::2]  
b = torch.randn(300)[::3]
```

修改其内存偏移量计算方式即可，需要传入三个 tensor 各自的 stride。

```
mask = tid < (M * N)  
i1 = tid % N  
i0 = tid // N  
*a = tl.load(a_ptr + i0 * a_stride0 + i1 * a_stride1, mask=mask)  
b = tl.load(b_ptr + i0 * b_stride0 + i1 * b_stride1, mask=mask)  
o = a + b  
*tl.store(o_ptr + i0 * o_stride0 + i1 * o_stride1, o, mask=mask)
```

解法二：支持 stride

假设 a, b 来自两个 2d tensor 上的不连续切片。

```
a = torch.randn(200, 200)[::2, ::2]
```

```
b = torch.randn(300, 300)[::3, ::3]
```

```
mask = tid < (M * N)
```

```
i1 = tid % N
```

```
i0 = tid // N
```

```
*a = tl.load(a_ptr + i0 * a_stride0 + i1 * a_stride1, mask=mask)
```

```
b = tl.load(b_ptr + i0 * b_stride0 + i1 * b_stride1, mask=mask)
```

```
o = a + b
```

```
*tl.store(o_ptr + i0 * o_stride0 + i1 * o_stride1, o, mask=mask)
```

有了各个维度的索引，以及 tensor 每个维度的 stride 就可以求出偏移量。

Q: 怎么把每个 tensor 的 stride 传给 JITFunction 呢？

A: Triton JITFunction 不接受 list tuple 等类型的参数，只能展开作为多个 int 来传。但是 tensor 的 rank 在编写 jit function 的时候是不确定的。无法写出一份适用所有情况的代码。

```
TypeError: Unsupported type <class 'tuple'> for (128, 128)
```

计算内存偏移量的代码模式化，所有的 pointwise 运算原理上具有相似性，可以采用**自动代码生成**的方式。

@pointwise_dynamic

@triton.jit

```
def add_kernel(a_ptr, b_ptr, o_ptr, N, TILE_SIZE: tl.constexpr):
```

```
    pid = tl.program_id(0)
```

```
    offsets = pid * TILE_SIZE + tl.arange(0, TILE_SIZE)
```

```
    mask = offsets < N
```

```
    a = tl.load(a_ptr + offsets, mask=mask)
```

```
    b = tl.load(b_ptr + offsets, mask=mask)
```

```
    o = a + b
```

```
    tl.store(o_ptr + offsets, o, mask=mask)
```

```
* def add(a, b):
```

```
    o = torch.empty_like(a)
```

```
    N = a.numel()
```

```
    TILE_SIZE = 512
```

```
    grid = triton.cdiv(N, TILE_SIZE)
```

```
    add_kernel[grid](a, b, o, N, TILE_SIZE)
```

```
    return o
```



```
@pointwise_dynamic(is_tensor=[True, True, False], promotion_methods=[(0, 1, "DEFAULT")])
```

```
@triton.jit
```

```
def add_func(x, y, alpha):
```

```
    return x + y * alpha
```

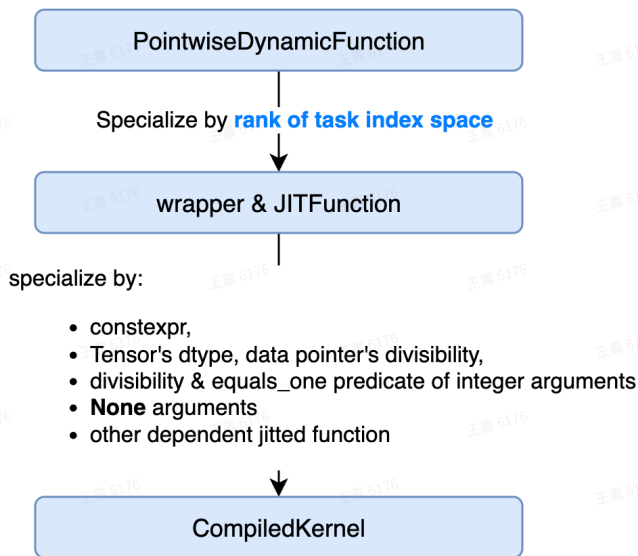
```
** def add(A, B, *, alpha=1):
```

```
    return add_func(A, B, alpha)
```

PointwiseDynamicFunction

给定需要执行的标量运算，自动生成能适用于 tensor 的 pointwise operation.

- 根据输入的 argument 来确定**任务索引空间的 rank**（各个 tensor broadcast 后的形状）；
- 根据 rank 按需生成特化版本的函数。
- 转发参数，调用适用的特化版函数。



一个 **PointwiseDynamicFunction** 可能对应若干个 wrapper 和 JITFunction. 此处有一层特化，由 **PointwiseDynamicFunction** 维护。

pointwise_dynamic 实现为一个装饰器，装饰在一个 JITFunction 上。得到一个 **PointwiseDynamicFunction**。

一个 **JITFunction** 可能对应若干个 **CompiledKernel**, 此处是另外一层特化，这是 triton 运行时维护的。

```
18 def add_func_wrapper_rank_2(in0: Union[torch.Tensor, StridedBuffer], in1: Union[torch.Tensor, StridedBuffer], val0, /, *, out0: Union[torch.Tensor, StridedBuffer]):
19     """Generated wrapper function with Pointwise: StridedBuffer, StridedBuffer, scalar, StridedBuffer(a1!) -> StridedBuffer(a1!)"""
20     assert in0.shape == in1.shape == out0.shape, 'operand shapes mismatch'
21     # task partitioning
22     shape = out0.shape
23     num_tasks = out0.numel()
24     if num_tasks == 0:
25         return out0
26     tile_sizes = heuristics_for_tile_size(512, *shape)
27     tile_size = math.prod(tile_sizes)
28     num_tiles = math.prod(triton.cdiv(size, tile_size) for size, tile_size in zip(shape, tile_sizes))
29     num_ctas = min(65536, num_tiles)
30     tiles_per_cta = triton.cdiv(num_tiles, num_ctas)
31     num_warps = heuristics_for_num_warps(tile_size)
32     one_tile_per_cta = tiles_per_cta==1
33     grid = (num_ctas, 1, 1)
34     # kernel launch
35     in0_strides = in0.stride()
36     in0_stride_order = stride_order(in0_strides)
37     in1_strides = in1.stride()
38     in1_stride_order = stride_order(in1_strides)
39     out0_strides = out0.stride()
40     out0_stride_order = stride_order(out0_strides)
41     with torch_device_fn.device(in0.device.index):
42         add_func_kernel_rank_2[grid](
43             in0, in1, val0, out0,
44             in0_strides[0], in0_strides[1], # stride for in0
45             in0_stride_order[0], in0_stride_order[1], # stride order for in0
46             in1_strides[0], in1_strides[1], # stride for in1
47             in1_stride_order[0], in1_stride_order[1], # stride order for in1
48             out0_strides[0], out0_strides[1], # stride for out0
49             out0_stride_order[0], out0_stride_order[1], # stride orderfor out0
50             shape[0], shape[1], # task indexing space
51             num_tasks, # num tasks
52             tiles_per_cta=tiles_per_cta, # tiles_per_cta
53             tile_size0=tile_sizes[0],
54             tile_size1=tile_sizes[1],
55             one_tile_per_cta=one_tile_per_cta,
56             num_warps=num_warps,
57         )
58     return out0
```

- 生成的wrapper代码
- 针对2d的输入进行代码生成
- 包括了tile、grid的划分，shape、stride等参数的传递以及启动kernel

- 生成的Triton kernel代码
- 根据stride进行数据的加载，处理不连续的情况

```
65 @Libentry()
66 @triton.jit(do_not_specialize=['val0'])
67 def add_func_kernel_rank_2(
68     in0_ptr: tl.tensor, # of tl.pointer_type
69     in1_ptr: tl.tensor, # of tl.pointer_type
70     val0,
71     out0_ptr: tl.tensor, # of tl.pointer_type
72     in0_stride0: int, in0_stride1: int, # strides for in0
73     in0_stride_order0: tl.constexpr, in0_stride_order1: tl.constexpr, # stride order for in0
74     in1_stride0: int, in1_stride1: int, # strides for in1
75     in1_stride_order0: tl.constexpr, in1_stride_order1: tl.constexpr, # stride order for in1
76     out0_stride0: int, out0_stride1: int, # strides for out0
77     out0_stride_order0: tl.constexpr, out0_stride_order1: tl.constexpr, # stride order for out0
78     s0: int, s1: int, # task_space
79     num_tasks: int,
80     tiles_per_cta: int,
81     tile_size0: tl.constexpr, tile_size1: tl.constexpr,
82     one_tile_per_cta: tl.constexpr,
83 ):
84     pid = tle.program_id(0)
85     num_tiles0 = tl.cdiv(s0, tile_size0)
86     num_tiles1 = tl.cdiv(s1, tile_size1)
87     if one_tile_per_cta: # monolithic kernel style
88         tile_id = pid
89         # pid multi index reconstruction: we use c ordering, right axes changes fastest
90         tile_id1 = tile_id % num_tiles1
91         tile_id //= num_tiles1
92         tile_id0 = tile_id
93
94         # tile offsets
95         offset0 = (tile_id0 * tile_size0).to(tl.int32)
96         offset1 = (tile_id1 * tile_size1).to(tl.int32)
97         # loads
98         in0_bptr = tl.make_block_ptr(in0_ptr, (s0, s1), (in0_stride0, in0_stride1), (offset0, offset1), (tile_size0, tile_size1), order=(in0_stride_order0, in0_stride_order1))
99         in0 = tl.load(in0_bptr, boundary_check=(in0_stride_order0, in0_stride_order1)).to(in0_ptr.type.element_ty) # workaround the bug on bool, we should use the original pointer's dtype (instead of block pointer's)
100         in1_bptr = tl.make_block_ptr(in1_ptr, (s0, s1), (in1_stride0, in1_stride1), (offset0, offset1), (tile_size0, tile_size1), order=(in1_stride_order0, in1_stride_order1))
101         in1 = tl.load(in1_bptr, boundary_check=(in1_stride_order0, in1_stride_order1)).to(in1_ptr.type.element_ty) # workaround the bug on bool, we should use the original pointer's dtype (instead of block pointer's)
102
103         # compute
104         out0 = add_func(in0, in1, val0)
105
106         # stores, note that store to block pointer does not automatically cast the value to the pointer's dtype
107         out0_bptr = tl.make_block_ptr(out0_ptr, (s0, s1), (out0_stride0, out0_stride1), (offset0, offset1), (tile_size0, tile_size1), order=(out0_stride_order0, out0_stride_order1))
108         tl.store(out0_bptr, out0.to(out0_bptr.type.element_ty), boundary_check=(out0_stride_order0, out0_stride_order1))
109     else: # grid-stride-loop style kernel
110         num_ctas = tle.num_programs(0)
111         for j in range(0, tiles_per_cta):
112             tile_id = pid + j * num_ctas
113             # pid multi index reconstruction: we use c ordering, right axes changes fastest
```

目录

01 Triton 算子开发示例

02 FlagGems简介

03 FlagGems特性 ---- 运行时优化

04 FlagGems contribute


```
@triton.jit
def add_kernel(x_ptr, y_ptr, output_ptr,
               n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    output = x + y
    tl.store(output_ptr + offsets, output, mask=mask)
```

jit function: jit function 本体。

使用 python 语法的子集，但这部分代码并不由 python 解释器执行，而是由 triton 编译器编译。

```
def add(x: torch.Tensor, y: torch.Tensor):
    output = torch.empty_like(x)
    n_elements = output.numel()
    grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']), )
    add_kernel[grid](x, y, output, n_elements, BLOCK_SIZE=1024)
    return output
```

wrapper: 一个用于处理 torch.Tensor 的函数，其中会调用 triton jit function。

其中 jit function 嵌入在 python 程序的解释执行中被编译和调用。
wrapper 的代码都是 python code。

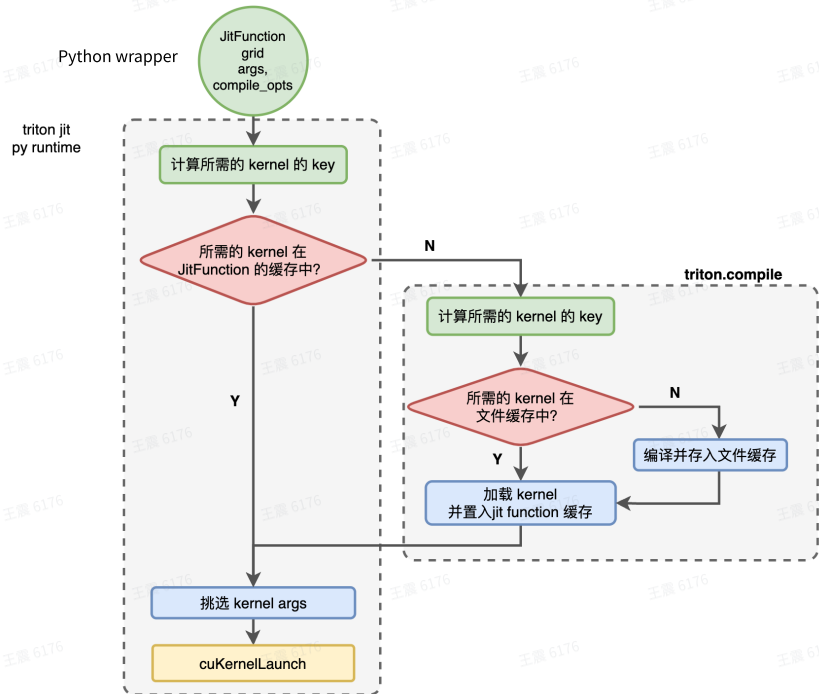
Triton 语言在 Python 中的主要使用方式是 jit(即时编译)，执行一个算子有：

Wrapper: 准备输入输出，计算grid等然后调用jit function

Triton jit Runtime:

- 计算特化信息：根据 jit function 和传入的具体参数提取确定一个 CompiledKernel 所需的信息；
- 触发 triton 编译。一般附带有 CodeCache 的 Runtime 会把编译结果缓存起来管理；
- 执行编译结果（比如 cubin 文件）。

Compiler: 编译出所需的 CompiledKernel，交由 Runtime 执行。



Triton 是 Python 的 DSL，所以用 Triton 来实现 Tensor 的算子时，最直接的做法是，Wrapper 和 JIT Runtime 都是实现在 Python 的。

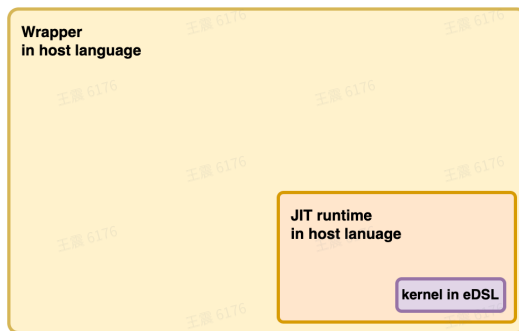
Pros: 快速开发且得到不错的性能；

Cons:

- Python 端的 Wrapper；
- Python 端的 JIT Runtime；
- Python 端的 CompiledKernel
 - 生成通过 C 函数：使用 cuda driver C API 调用
 - 提供 binding 供 python 调用。

实际使用的过程中，很有可能 Wrapper + JIT Runtime 这两层的开销远大于用 DSL 写出来的 kernel 本身。

可能的时间占比



C++ wrapper

FlagGems中提供了c++ wrapper以优化triton运行时开销

Wrapper: 将原本python端的wrapper在C++端重新实现, kernel仍在python端

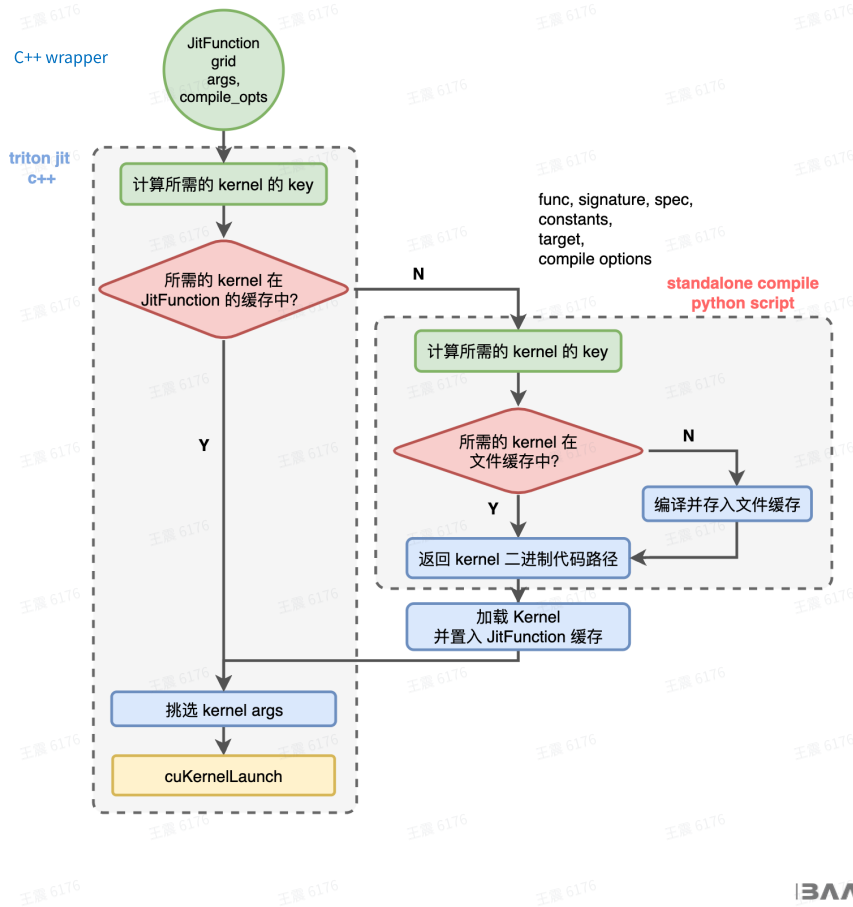
Triton jit Runtime:

- 在C++端重新实现
- TritonJITFunction:** 和 python 端 JitFunction 对应, 是所有 jit function 实例的类型, 每个实例管理 key 到 TritonKernel 的缓存。负责参数分发和 key 计算, kernel 调用。
- TritonKernel:** 对编译结果(cubin)的简单封装, 和 python 端的 CompiledKernel 对应。

Compiler:

standalone_compiler: 仍然复用 python 端的 triton.compile, 但使用一个单独的编译脚本。

- 需要由 Runtime 提供函数本体(fn), 签名(signature), 常数参数(constants), 特化信息 (spec), 编译目标 (target), 编译选项(compile_options)等信息。
- 编译 cubin 并且返回路径给 Runtime。



```
8 namespace flag_gems {
9 using namespace triton_jit;
10
11 at::Tensor add_tensor(const at::Tensor &a, const at::Tensor &b) {
12
13     at::ScalarType out_dtype = at::promote_types(a.scalar_type(), b.scalar_type());
14     at::Tensor out = at::empty(a.sizes(), at::TensorOptions().dtype(out_dtype).device(a.device()));
15
16     const TritonJITFunction &f =
17         TritonJITFunction::getInstance(std::string(utils::get_triton_src_path() / "add.py"),
18                                         "add_kernel");
19
20     // add utility to build this automatically
21     int64_t tile_size = 1024;
22     const int num_warps = 8;
23     const int num_stages = 1;
24     int64_t n = out.numel();
25     const unsigned int num_blocks = (n + tile_size - 1) / tile_size;
26
27     // getCurrentCUDASTream ensures that the stream is initialized, a default stream for each device
28     c10::cuda::CUDASTream stream = c10::cuda::getCurrentCUDASTream();
29     c10::DeviceGuard guard(out.device());
30     CUstream raw_stream = static_cast<CUstream>(stream.stream());
31     f(raw_stream, num_blocks, 1, 1, num_warps, num_stages, a, b, out, n, tile_size);
32     return out;
33 }
34 } // namespace flag_gems
```

调用 TritonJITFunction 之外的部分和写别的算子没有区别

测试环境: torch 2.6.0, triton 3.2.0, cuda 12.6, RTX-3090

Case:

add: (128k,) 的 fp32 的 vector_add
sum_dim: (16, 4k) 的 fp32 tensor, sum_dim=1

名称	使用方式
libtorch (cpp)	对于 torch 内建算子, 直接使用 at::op_name 接口, 如 at::add
torch (python)	对于 torch 内建算子, 直接使用 torch.op_name 接口, 如 torch.add
triton (python)	直接使用 triton 函数以及 python wrapper, triton_jit_function[grid](...)
triton_torch_ops (python)	将上述函数注册到 torch dispatcher, 然后通过 torch.ops.lib_name.op_name 调用
triton_cpp (cpp)	通过 C++ wrapper 以及 Triton JIT C++ 运行时使用 triton 函数
triton_cpp_torch_ops (python)	将上述函数注册到 torch dispatcher, 然后通过 torch.ops.lib_name.op_name 调用

Wrapper 时间（粗略估计为两次 launch 时间差）对比

	libtorch (cpp)	torch (python)	triton (python)	triton_torch_ops (python)	triton_cpp (cpp)	triton_cpp_torch_ops (python)
add	14.44us	14.44us	61.11us	103.33us	14.44us	17.78us
sum_dim	17.78us	18.89us	82.22us	140.00us	16.67us	24.44us

注：torch.op 的调用方式比torch.ops.lib.op 的方式更高效。因为这样的调用方式是 unbox 的，但实现较为麻烦。torch 是用 template-based codegen 生成的

Kernel 时间对比

	torch	triton
add	2.126us	2.097us
sum_dim	8.805us	4.538us

目录

01 Triton 算子开发示例

02 FlagGems简介

03 FlagGems特性

04 FlagGems contribute

github.com/FlagOpen/FlagGems/projects?query=is%3Aopen

工作 学习 工具 Quick Reference &...

FlagOpen / FlagGems

Type to search

<> Code Issues 52 Pull requests 39 Discussions Actions **Projects 1** Wiki Security Insights

Projects
Templates

Welcome to projects

Built like a spreadsheet, project tables give you a live canvas to filter, sort, and group issues and pull requests. Tailor them to your needs with custom fields and saved views.

Learn more

is:open

Link a project

1 Open 0 Closed

Sort

Triton China Community
#3 updated 18 minutes ago

本仓库是Triton中国社区贡献者工作空间。用于沟通与协作事项记录。欢迎你参与进来！ The workspace for contributors to the Triton China community, used for communication and collaboration records. You are welcome to participate!

Todo 18 / 100 Estimate: 0

This item hasn't been started

FlagGems #884
Code Contribution: [Easy] [Operator Development] acosh

FlagGems #885
Code Contribution: [Easy] [Operator Development] addcdv

FlagGems #886
Code Contribution: [Easy] [Operator Development] addcmul

FlagGems #887
Code Contribution: [Easy] [Operator Development] celu

FlagGems #888
Code Contribution: [Medium] [Operator Development] addmv

FlagGems #890
Code Contribution: [Medium] [Operator Development] logspace

FlagGems #891
Code Contribution: [Medium] [Operator Development] logsumexp

FlagGems #889
Code Contribution: [Medium] [Operator Development] addr

FlagGems #892
Code Contribution: [Hard] [Operator Development] index_fill

FlagGems #893
Code Contribution: [Hard] [Operator Development] index_reduce

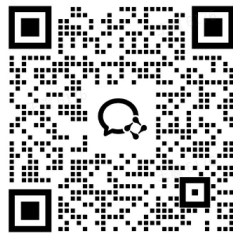


感谢垂听！

欢迎访问开源社区：

<https://github.com/FlagOpen/FlagGems>

Triton中国社区微信群



扫码添加微信，私信“加群”